



Informe Técnico Ejecutivo

DiabetesNHANES - Pipeline de Ciencia de Datos

Evaluación Final Transversal - SCY1101 Programación para la Ciencia de Datos

Equipo: Manuel Díaz, Claudio Aro, Guillermo Cerda

Repositorio: github.com/apotheosisss/PCS_EvParcial3

Tabla de contenidos

1. Resumen ejecutivo

DiabetesNHANES es una solución de ciencia de datos de extremo a extremo que estima el riesgo de diabetes en población adulta a partir de datos públicos de la encuesta NHANES 2021-2023 (CDC/NCHS). El sistema integra múltiples fuentes, ejecuta un pipeline ETL reproducible con Kedro, entrena un portafolio de seis modelos de machine learning y expone los resultados mediante una API REST y un dashboard interactivo, todo containerizado con Docker y desplegable en la nube (AWS).

Sobre 8.149 participantes adultos, el modelo seleccionado (regresión logística calibrada) alcanza un ROC-AUC de 0,81 y un PR-AUC de 0,44, con una prevalencia real de diabetes del 16 %. El valor de negocio es un instrumento de tamizaje temprano y educativo que prioriza el recall (detectar casos) sobre la exactitud global, adecuado a un problema de salud pública con clases desbalanceadas.

Aviso: la variable objetivo es una aproximación analítica/educativa basada en criterios ADA; no constituye diagnóstico clínico.

2. Contexto de negocio

- **Problema:** la diabetes tipo 2 suele diagnosticarse tarde; un tamizaje temprano de bajo costo permite intervenir sobre factores de riesgo modificables.
- **Objetivo:** estimar el riesgo/estado de diabetes usando variables demográficas, antropométricas, de estilo de vida y biomarcadores disponibles en NHANES.
- **Impacto esperado:** priorizar población para chequeos, dimensionar el problema por segmentos y ofrecer un simulador individual de riesgo.

3. Arquitectura técnica

El proyecto usa Kedro como framework de orquestación: catálogo de datos declarativo (las rutas viven en catalog.yml, no en el código) y el flujo dividido en pipelines independientes y reproducibles.

Capa	Componente	Tecnología
Ingesta	8 archivos XPT + umbrales + auditoría	Kedro, pandas, pyreadstat
Limpieza/Features	merge SEQN, imputación, target, encoding	pandas, numpy
Modelado	6 clasificadores + calibración	scikit-learn, xgboost, lightgbm, catboost
Servicio	API REST + dashboard	FastAPI, Next.js
Infraestructura	containers + despliegue	Docker Compose, AWS EC2, GitHub Actions

Pipelines Kedro: ingestion -> cleaning -> feature_engineering -> modeling -> reporting. Cada uno se ejecuta con kedro run y produce artefactos versionados por capa (data/01_raw ... data/08_reporting).

4. Pipeline ETL y fuentes de datos

Integra tres tipos de fuentes, cumpliendo el requisito de integración de múltiples fuentes:

- **NHANES XPT (SAS transport):** DEMO, DIQ, BMX, GHB, GLU, PAQ, SLQ, BPXO del ciclo 2021-2023.

- **Archivo propio de umbrales (CSV):** criterios clínicos parametrizados (A1C, glucosa, IMC, edad).
- **Base SQLite de auditoría:** tablas ingestion_audit y etl_audit que registran filas, columnas y nulos antes/después por corrida.

Robustez: la unión se hace por la llave SEQN con validación que aborta si hay SEQN duplicado; validate_required_columns lanza error si falta una columna obligatoria; los códigos especiales NHANES (7, 9, 77, 99) se convierten a nulo antes de imputar.

5. Transformaciones avanzadas y optimización

Se aplican técnicas optimizadas para gran escala (módulo utils/transforms.py), con impacto medido sobre el dataset real:

Técnica	Método	Resultado
Optimización memoria	downcast dtypes + category	2,74 MB -> 0,76 MB (-72 %)
Broadcasting	z-score vectorizado NumPy	estandarización sin bucles
Pivot	pivot_table agregado	prevalencia por segmento
Reshape	melt wide->long	8149x44 -> 16298x3
Chunking	lectura por bloques	memoria O(grupos)

6. Limpieza y construcción del target

La variable diabetes_target se construye ANTES de imputar sus fuentes para no inventar positivos: vale 1 si DIQ010 = 1 (diagnóstico reportado), o LBXGH >= 6,5 (A1C), o LBXGLU >= 126 (glucosa en ayunas); 0 en caso contrario. Las filas sin ninguna fuente determinable se descartan en lugar de asumir un 0.

Decisión clave (revisión de calidad): se detectó y corrigió una fuga de datos excluyendo los biomarcadores que definen el target (LBXGH, LBXGLU y derivados) del conjunto de features, evitando métricas infladas artificialmente.

7. Portafolio de modelos ML

Se entrenaron y compararon seis clasificadores dentro de un Pipeline ajustado solo con train (sin fuga), con class_weight balanceado y selección por PR-AUC, métrica honesta ante clases desbalanceadas (prevalencia 16 %).

Modelo	Accuracy	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression *	0,723	0,747	0,463	0,816	0,441
Random Forest	0,765	0,678	0,480	0,811	0,403
Gradient Boosting	0,742	0,705	0,466	0,814	0,423
XGBoost	0,734	0,690	0,454	0,808	0,414
LightGBM	0,758	0,590	0,438	0,797	0,395
CatBoost	0,748	0,655	0,454	0,807	0,421

* Modelo seleccionado (mayor PR-AUC). Calibrado con isotonic regression, umbral óptimo 0,31, Brier 0,11.

Interpretación: la regresión logística gana en PR-AUC y recall, detectando ~75 % de los casos positivos; el resto de modelos ofrece mayor accuracy pero menor recall, poco útil cuando el costo de no detectar un caso es alto.

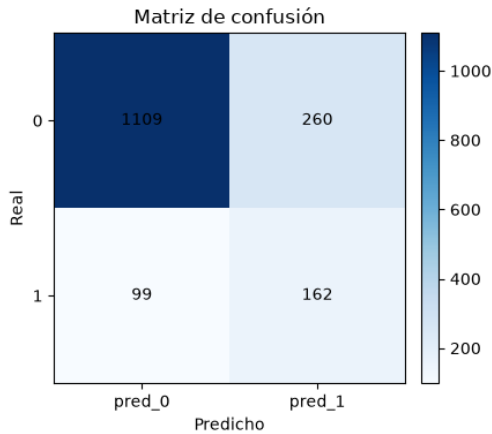


Figura 1. Matriz de confusión (umbral 0,31): TN=1109, FP=260, FN=99, TP=162.

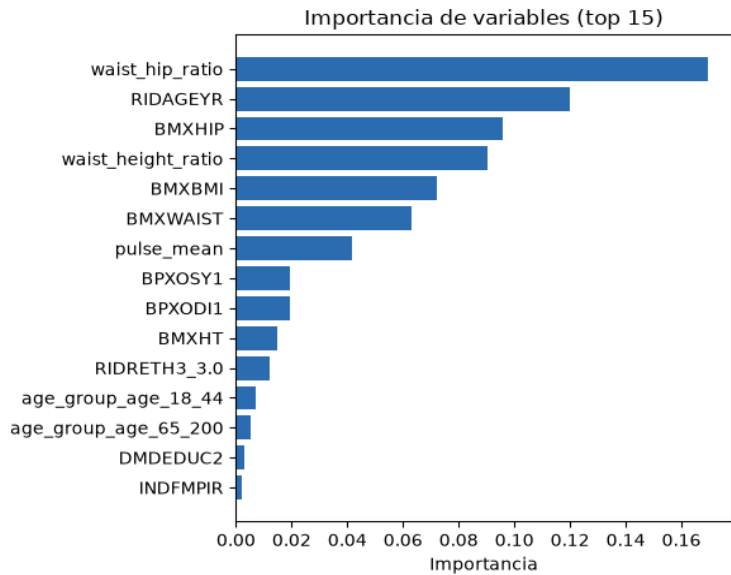


Figura 2. Importancia de variables del modelo.

8. API REST y dashboard

La API FastAPI expone salud, metadatos, métricas y predicción (individual y por lote), documentada automáticamente en /docs. Si el modelo no existe responde 503 en lugar de caerse.

Endpoint	Método	Descripción
/health	GET	estado del servicio y del modelo
/model-info	GET	nombre y versión del modelo
/metrics	GET	métricas del modelo entrenado
/predict	POST	predicción de riesgo individual
/predict/batch	POST	predicción por lote

El dashboard (Next.js) ofrece tres vistas por audiencia: Ejecutiva (KPIs y distribuciones), Técnica (métricas, matriz de confusión, importancia) y Operativa (filtros, descarga CSV y simulador de predicción).

9. Contenerización, CI/CD y despliegue

- **Docker:** cuatro servicios (kedro-etl, api, dashboard, db) orquestados con docker-compose; depends_on para el orden, volumen data/ compartido e imágenes python:3.11-slim optimizadas por capas.
- **CI/CD:** workflow de GitHub Actions (.github/workflows/ci.yml) que corre lint, tests (pytest), build del frontend y build de la imagen Docker en cada push/PR a develop y main.
- **AWS:** desplegado y verificado end-to-end en una instancia EC2 (Amazon Linux 2023, t3.large) de un AWS Academy Learner Lab, siguiendo docs/guia_despliegue_aws.md. Durante el despliegue real se detectaron y corrigieron dos problemas de infraestructura, documentados como evidencia de troubleshooting en el Anexo D.

10. KPIs y análisis de resultados

KPI	Valor
Participantes analizados	8.149
Prevalencia de diabetes (target)	16,0 %
Recall del modelo (casos detectados)	~75 %
ROC-AUC / PR-AUC	0,81 / 0,44
Reducción de memoria (ETL)	-72 %
Modelos comparados	6

11. Colaboración y gestión de proyecto

El equipo se organizó en tres ramas de responsabilidad (feature/a, feature/b, feature/c), cada una con un integrante a cargo:

Integrante	Rama	Responsabilidad
Manuel Díaz	feature/a	Arquitectura Kedro, catálogo de datos e ingesta de las fuentes NHANES.
Guillermo Cerda	feature/b	Limpieza, unión por SEQN, construcción del target y features derivadas.
Claudio Aro	feature/c	Modelado final, calibración, API REST, dashboard, Docker y despliegue en AWS.

Estrategia Git Flow simplificada: main estable, develop de integración y una rama feature por responsabilidad. Ninguna rama feature se integró directo a main; todo pasó por Pull Request a develop con revisión. Se registraron 8 Pull Requests y un revert (PR #4) que evidencia que el control de calidad detectó y corrigió un merge incorrecto.

La colaboración se apoyó en un contrato de interfaz (docs/CONTRATO_FEATURE_B.md) que fijó las columnas del dataset entre ramas, permitiendo desarrollar modelado, API y dashboard en paralelo.

12. Limitaciones y mejoras futuras

- El target es una etiqueta educativa (criterios ADA), no un diagnóstico clínico; algunas variables son autoinformadas.
- NHANES es una encuesta poblacional transversal, no una historia clínica longitudinal.
- Mejoras: CI que bloquee merges con tests en rojo, build multi-stage, monitoreo de data drift y reentrenamiento programado.

Anexos

A. Estructura del repositorio

/src (pipelines Kedro), /api (FastAPI), /frontend (Next.js), /dashboards, /docker, /docs, /tests, /repo (evidencias Git), /data, /scripts, /deploy/aws.

B. Reproducibilidad

```
cp .env.example .env -> python scripts/download_nhanes.py -> kedro run -> docker compose -f
docker/docker-compose.yml up --build
```

C. Evidencias de pruebas

Suite pytest sobre ingesta, limpieza, features, modelo y transformaciones avanzadas; ejecutada localmente y en CI (GitHub Actions).

D. Evidencia de despliegue en AWS

Instancia verificada: EC2, AMI Amazon Linux 2023, tipo t3.large (8 GB RAM, 2 vCPU), 20 GB gp3, dentro de un AWS Academy Learner Lab. Desplegada con deploy/aws/user_data.sh (bootstrap: Docker, Compose, Buildx y clonado del repo) y deploy/aws/deploy.sh (descarga NHANES, ETL y levantamiento de servicios).

Contenedores verificados en ejecución:

Servicio	Estado verificado
nhanes-db	Up (healthy)
nhanes-api	Up (healthy)
nhanes-frontend	Up (healthy)

Verificación de la API sobre la instancia desplegada:

```
GET /health ->
{"status":"ok","model_loaded":true,"metrics_loaded":true,"model_version":"v3.0.0"}

GET /model-info ->
{"model_name":"logistic_regression","model_version":"v3.0.0","n_features":42}
```

Troubleshooting real durante el despliegue:

- **Docker Buildx faltante:** el AMI de Amazon Linux 2023 no incluye el plugin Buildx, requerido por “docker compose build” en versiones recientes de Compose. Se detectó al fallar el build del servicio kedro-etl con el error “compose build requires buildx 0.17.0 or later”. Solución: se agregó la instalación de Buildx a user_data.sh, descargando el binario oficial desde GitHub Releases.
- **Key pair “vockey” no descargable:** en el Learner Lab usado, la private key de “vockey” no estaba disponible para descarga desde la consola EC2 (solo se puede descargar una key al momento de crearla, y “vockey” ya viene preexistente). Solución: se creó una key pair propia y se relanzó la instancia con ella; documentado en docs/guia_despliegue_aws.md.